

NAME

Parsers::SimpleCalcParser

SYNOPSIS

```
use Parsers::SimpleCalcParser ;

use Parsers::SimpleCalcParser qw(:all);
```

DESCRIPTION

Parsers::SimpleCalcParser class provides the following methods:

new, yyclearin, yyerrok, yyerror, yyparse

Parsers::SimpleCalcParser.yy parser grammar definition file implements a simple calculator and is provided to highlight usage of lexer capability available through Parsers::SimpleCalcYYLexer, which in turn uses Parsers::YYLexer and Parsers::Lexer classes to provide underlying lexer functionality.

The parser package and token table files, Parsers::SimpleCalcParser.pm and SimpleCalcParser.tab.ph, are automatically generated from parser grammar definition file, Parsers::SimpleCalcParser.yy, using byacc available through perl-byacc1.8 modified with perl5-byacc-patches-0.5 for generation of object oriented parser:

```
byacc -l -P -d -b SimpleCalcParser SimpleCalcParser.yy
mv SimpleCalcParser.tab.pl SimpleCalcParser.pm
```

METHODS**new**

```
$SimpleCalcParser = new Parsers::SimpleCalcParser($YYLex,
\&Parsers::SimpleCalcParser::yyerror);
$SimpleCalcParser = new Parsers::SimpleCalcParser($YYLex,
\&Parsers::SimpleCalcParser::yyerror, $Debug);
```

Using specified YYLex YYError functions, new method generates a new SimpleCalcParser and returns a reference to newly created SimpleCalcYYParser object.

Examples:

```
# Input string...
$InputText = "3 + 4 +6\nx=3\nny=5\nx+y\nx+z\n";
$YYLexer = new Parsers::SimpleCalcYYLexer($InputText);
$YYLex = $YYLexer->GetYYLex();

$Debug = 0;
$SimpleCalcParser = new Parsers::SimpleCalcParser($YYLex,
\&Parsers::SimpleCalcParser::yyerror, $Debug);
$Value = $SimpleCalcParser->yyparse();
print "Value = " . (defined($Value) ? "$Value" : "Undefined") . "\n";

# Input file...
$InputFile = "TestSimpleCalcParser.txt";
open INPUTFILE, "$InputFile" or die "Couldn't open $InputFile: $!\n";

$YYLexer = new Parsers::SimpleCalcYYLexer(\*INPUTFILE);
$YYLex = $YYLexer->GetYYLex();

$Debug = 0;
$SimpleCalcParser = new Parsers::SimpleCalcParser($YYLex,
\&Parsers::SimpleCalcParser::yyerror, $Debug);
$Value = $SimpleCalcParser->yyparse();
print "Value = " . (defined($Value) ? "$Value" : "Undefined") . "\n";

close INPUTFILE;

# Input iterator...
$InputFile = "TestSimpleCalcParser.txt";
open INPUTFILE, "$InputFile" or die "Couldn't open $InputFile: $!\n";
$InputIterator = sub { return <INPUTFILE>; };
```

```
$YYLexer = new Parsers::SimpleCalcYYLexer($InputIterator);
$YYLex = $YYLexer->GetYYLex();

$Debug = 0;
$SimpleCalcParser = new Parsers::SimpleCalcParser($YYLex,
    \&Parsers::SimpleCalcParser::yyerror, $Debug);
$Value = $SimpleCalcParser->yyparse();
print "Value = " . (defined($Value) ? "$Value" : "Undefined") . "\n";

close INPUTFILE;
```

yyclearin

```
$SimpleCalcParser->yyclearin();
```

yyclearin method clears any previous look-ahead token after encountering a syntax error during parsing. It can be used after yyerrok in a grammar rule with the reserved word error.

yyerrok

```
$SimpleCalcParser->yyerrok();
```

yyerrok method is used with the reserved word error in grammar rule to indicate error recovery is complete after encountering a syntax error during parsing.

yyerror

```
$SimpleCalcParser->yyerror();
```

yyerror function is provided for the caller to use during initialization of a parser. It is used by yyparse to print any error messages encountered during parsing of the input.

yyparse

```
$Value = $SimpleCalcParser->yyparse();
```

Returns *Value* after parsing all the input from an input stream using specified grammar rules.

AUTHOR

Manish Sud <msud@san.rr.com>

SEE ALSO

Lexer.pm, YYLexer.pm, SimpleCalcYYLexer.pm

COPYRIGHT

Copyright (C) 2026 Manish Sud. All rights reserved.

This file is part of MayaChemTools.

MayaChemTools is free software; you can redistribute it and/or modify it under the terms of the GNU Lesser General Public License as published by the Free Software Foundation; either version 3 of the License, or (at your option) any later version.